

RAG Document Q&A System

Complete Build Guide — Architecture, Schema, Code & Deployment

Project 1 of 3 — Resume Project Series

Stack: Python · FastAPI · LangChain · Qdrant · OpenAI

Prepared for Saksham Tiwari

Table of Contents

Part A — Understanding

1. What We're Building	3
2. System Architecture	4
3. Resume Bullet Mapping	5

Part B — Planning

4. Tech Stack & Why Each Piece	6
5. Folder Structure	7
6. Data & Schema Design	8
7. API Endpoint Specification	9

Part C — Building, Step by Step

8. Project Setup & Environment	10
9. Document Ingestion Pipeline	11
10. Chunking Strategy	12
11. Embeddings & Qdrant Storage	13
12. The Retrieval Chain	14
13. Answer Generation & Prompting	15
14. FastAPI Routes	16
15. Chat History & Sessions	17
16. Error Handling & Validation	18
17. Wiring It Together (main.py)	19

Part D — Shipping It

18. Docker & Docker Compose	20
19. Testing the API	21
20. Build Order Checklist	22
21. Bugs You Will Hit	23
22. Interview Q&A	24
23. Cheatsheet	25

PART A

Understanding

1. What We're Building

A **RAG (Retrieval-Augmented Generation) Document Q&A System**: users upload documents (PDFs to start), the system indexes them, and users can then ask natural-language questions that get answered *grounded in the document content* — not from the LLM's general knowledge, and not hallucinated.

Why RAG instead of just asking the LLM directly?

LLMs don't know your private documents, and they hallucinate confidently when they don't know something. RAG fixes both: it retrieves the actual relevant text chunks first, then asks the LLM to answer *using only those chunks*. The answer is traceable back to source text.

Core capabilities

- Upload one or more PDF documents
- Documents get parsed, split into chunks, embedded, and stored in a vector database
- User asks a question in plain English
- System finds the most relevant chunks via semantic (vector) search
- LLM generates an answer using those chunks as context, with source citations (page numbers / chunk references)
- Follow-up questions maintain conversation context (chat history)

What makes this a strong resume project

- Every GenAI team asks about RAG in interviews — you'll have a real, defensible answer
- Touches the full pipeline: ingestion, embeddings, vector search, prompt engineering, API design
- Directly reusable knowledge for Project 2 (NL-to-SQL) — same LangChain + OpenAI setup, different retrieval target

2. System Architecture

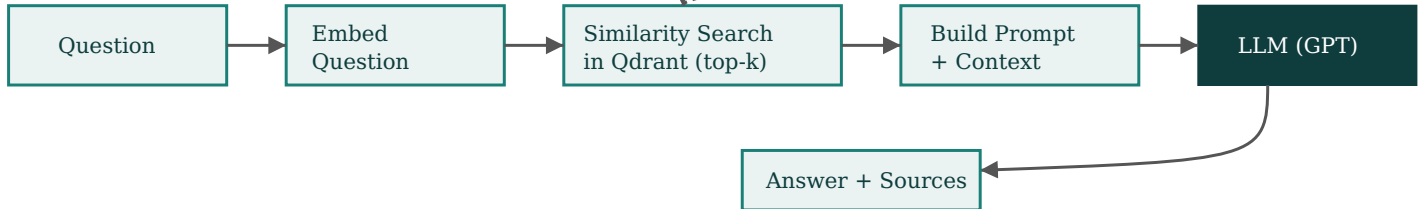
Two independent flows: **ingestion** (happens once per document) and **query** (happens per question). They share the same vector store.

INGESTION (one-time per document)



same vector store

QUERY (per user question)



Flow explained

Ingestion: a PDF is uploaded → text is extracted page by page → the text is split into overlapping chunks (~500-1000 tokens each) → each chunk is converted into a vector embedding → the embedding, along with metadata (document id, page number, chunk text), is stored in Qdrant.

Query: the user's question is embedded using the same embedding model → Qdrant performs a cosine-similarity search to find the top-k most relevant chunks → those chunks are inserted into a prompt template → the LLM generates an answer constrained to that context → the response includes the answer plus which chunks/pages it came from.

Key insight for interviews: the LLM never "reads" your whole document. It only ever sees the top-k chunks retrieved for that specific question. This is why chunking strategy and embedding quality matter more than model choice.

3. Resume Bullet Mapping

Map every technical decision to something you can defend in an interview.

Resume claim	What actually proves it
Built a RAG pipeline with LangChain	Document loaders, text splitters, and retrieval chains implemented via LangChain in <code>ingestion.py</code> and <code>chain.py</code>
Vector search with Qdrant	Qdrant collection with cosine similarity, top-k retrieval, metadata filtering by document id
Reduced hallucination via grounded generation	Prompt template explicitly instructs the model to answer only from provided context and say "not found in document" otherwise
REST API with FastAPI	Async endpoints for upload, ask, and session history with Pydantic validation
Source-cited answers	Each answer returns the chunk text + page number it was derived from
Multi-turn conversation support	Session-based chat history passed into the chain as conversational context

Suggested resume name: **DocuMind** or **ClarityDocs** — both sound like a real product rather than a tutorial project. Feel free to keep "RAG Document Q&A System" if you'd rather be literal — up to you.

PART B

Planning

4. Tech Stack & Why Each Piece

Layer	Choice	Why
API framework	FastAPI	Async by default, automatic OpenAPI docs, Pydantic validation — industry standard for Python AI services
Orchestration	LangChain	Handles document loaders, text splitters, retrieval chains — saves you from wiring embeddings/prompts manually
Vector DB	Qdrant	Purpose-built vector search, easy local Docker setup, filtering by metadata (doc id, page), free and open source
Embeddings	OpenAI <code>text-embedding-3-small</code>	Cheap, fast, good quality; swappable for a local model later (e.g. via Ollama) if you want a zero-cost demo
LLM	OpenAI <code>gpt-4o-mini</code>	Good quality-to-cost ratio for a resume project; swappable
PDF parsing	PyPDF / pdfplumber	Reliable text + page-number extraction
Metadata store	PostgreSQL (optional but recommended)	Track uploaded documents, sessions, chat history — Qdrant is not meant for this
Containerization	Docker + docker-compose	Qdrant + Postgres + API all spin up with one command — also what interviewers expect to see

Budget note: OpenAI embeddings + gpt-4o-mini are cheap (a full demo costs cents), but if you want a genuinely \$0 version for public demo purposes, swap embeddings and LLM for a local Ollama model. The LangChain code barely changes — only the model provider class changes. Mention this flexibility in interviews; it shows you understand the abstraction, not just one API.

5. Folder Structure

```
rag-doc-qa/
├── app/
│   ├── main.py           # FastAPI app entrypoint
│   ├── config.py         # env vars, settings
│   └── routes/
│       ├── upload.py     # POST /documents
│       ├── ask.py        # POST /ask
│       └── sessions.py   # GET /sessions/{id}
├── core/
│   ├── ingestion.py      # PDF -> chunks -> embeddings -> Qdrant
│   ├── chunking.py       # text splitter logic
│   ├── retriever.py      # Qdrant similarity search
│   ├── chain.py          # LangChain retrieval + generation chain
│   └── prompts.py        # prompt templates
├── db/
│   ├── postgres.py       # sqlalchemy engine/session
│   └── models.py         # Document, Session, Message tables
├── schemas/
│   └── schemas.py        # Pydantic request/response models
├── deps.py               # FastAPI dependency injection (db session, etc.)
├── tests/
│   ├── test_upload.py
│   ├── test_ask.py
│   └── test_chunking.py
├── docker-compose.yml
├── Dockerfile
├── requirements.txt
├── .env.example
└── README.md
```

6. Data & Schema Design

Qdrant collection: `document_chunks`

Field	Type	Notes
id	UUID	chunk id
vector	float[1536]	embedding (dimension depends on model)
payload.document_id	UUID	links back to Postgres <code>documents</code> table
payload.page_number	int	for source citation
payload.chunk_text	string	the actual chunk, returned alongside answers
payload.chunk_index	int	position within document

PostgreSQL tables

```
documents
  id          UUID PK
  filename    TEXT
  uploaded_at TIMESTAMP
  status      TEXT    -- 'processing' | 'ready' | 'failed'
  page_count  INT

sessions
  id          UUID PK
  document_id UUID FK -> documents.id
  created_at  TIMESTAMP

messages
  id          UUID PK
  session_id  UUID FK -> sessions.id
  role       TEXT    -- 'user' | 'assistant'
  content    TEXT
  sources    JSONB    -- chunk_ids / page numbers cited
  created_at  TIMESTAMP
```

Why split Qdrant and Postgres? Qdrant is optimized for vector similarity, not relational queries or transactional writes. Postgres tracks "what documents exist, what sessions exist, what was said" — the kind of structured, relational bookkeeping Qdrant isn't built for. This separation is itself a good interview talking point.

7. API Endpoint Specification

Method	Path	Purpose
POST	/documents	Upload a PDF, kicks off ingestion (can be async/background)
GET	/documents/{id}	Check ingestion status
POST	/sessions	Start a new Q&A session tied to a document
POST	/ask	Ask a question within a session
GET	/sessions/{id}/messages	Retrieve chat history

POST /ask — request/response

```
// Request
{
  "session_id": "uuid",
  "question": "What is the termination clause in this contract?"
}

// Response
{
  "answer": "The contract can be terminated with 30 days written notice...",
  "sources": [
    { "page": 4, "chunk_text": "Either party may terminate this agreement..." },
    { "page": 5, "chunk_text": "Notice must be provided in writing..." }
  ],
  "session_id": "uuid"
}
```

PART C

Building, Step by Step

8. Project Setup & Environment

```
mkdir rag-doc-qa && cd rag-doc-qa
python3.11 -m venv venv
source venv/bin/activate

pip install fastapi uvicorn[standard] langchain langchain-openai \
  langchain-community qdrant-client pypdf sqlalchemy \
  psycpg2-binary python-multipart pydantic-settings python-dotenv
```

.env.example

```
OPENAI_API_KEY=sk-...
QDRANT_URL=http://localhost:6333
DATABASE_URL=postgresql://postgres:postgres@localhost:5432/ragdb
EMBEDDING_MODEL=text-embedding-3-small
CHAT_MODEL=gpt-4o-mini
```

app/config.py

```
from pydantic_settings import BaseSettings

class Settings(BaseSettings):
    openai_api_key: str
    qdrant_url: str = "http://localhost:6333"
    database_url: str
    embedding_model: str = "text-embedding-3-small"
    chat_model: str = "gpt-4o-mini"

    class Config:
        env_file = ".env"

settings = Settings()
```

9. Document Ingestion Pipeline

Extract text page-by-page (keeping page numbers is what enables source citation later).

```
# app/core/ingestion.py
from pypdf import PdfReader

def extract_pages(file_path: str) -> list[dict]:
    reader = PdfReader(file_path)
    pages = []
    for i, page in enumerate(reader.pages):
        text = page.extract_text() or ""
        if text.strip():
            pages.append({"page_number": i + 1, "text": text})
    return pages
```

Gotcha: scanned PDFs (images, not real text) will return empty strings from `extract_text()`. For a resume demo, test with text-based PDFs. If you want to handle scanned documents too, mention OCR (pytesseract) as a documented extension rather than building it now — scope control matters.

10. Chunking Strategy

Chunk size is a real tradeoff: too small and you lose context; too large and irrelevant text dilutes the embedding and eats your top-k budget.

```
# app/core/chunking.py
from langchain.text_splitter import RecursiveCharacterTextSplitter

splitter = RecursiveCharacterTextSplitter(
    chunk_size=800,
    chunk_overlap=150,
    separators=["\n\n", "\n", ". ", " ", ""],
)

def chunk_pages(pages: list[dict]) -> list[dict]:
    chunks = []
    for page in pages:
        for idx, chunk_text in enumerate(splitter.split_text(page["text"])):
            chunks.append({
                "chunk_text": chunk_text,
                "page_number": page["page_number"],
                "chunk_index": idx,
            })
    return chunks
```

Why `RecursiveCharacterTextSplitter` ? It tries to split on paragraph breaks first, then sentences, then words — so chunks stay semantically coherent instead of cutting mid-sentence. The 150-token overlap prevents losing context that straddles a chunk boundary.

11. Embeddings & Qdrant Storage

```
# app/core/ingestion.py (continued)
from langchain_openai import OpenAIEmbeddings
from qdrant_client import QdrantClient
from qdrant_client.models import PointStruct, VectorParams, Distance
import uuid

embeddings = OpenAIEmbeddings(model=settings.embedding_model)
qdrant = QdrantClient(url=settings.qdrant_url)

COLLECTION = "document_chunks"

def ensure_collection():
    if not qdrant.collection_exists(COLLECTION):
        qdrant.create_collection(
            collection_name=COLLECTION,
            vectors_config=VectorParams(size=1536, distance=Distance.COSINE),
        )

def store_chunks(document_id: str, chunks: list[dict]):
    ensure_collection()
    texts = [c["chunk_text"] for c in chunks]
    vectors = embeddings.embed_documents(texts)

    points = [
        PointStruct(
            id=str(uuid.uuid4()),
            vector=vectors[i],
            payload={
                "document_id": document_id,
                "page_number": chunks[i]["page_number"],
                "chunk_index": chunks[i]["chunk_index"],
                "chunk_text": chunks[i]["chunk_text"],
            },
        )
        for i in range(len(chunks))
    ]
    qdrant.upsert(collection_name=COLLECTION, points=points)
```

12. The Retrieval Chain

```
# app/core/retriever.py
from qdrant_client.models import Filter, FieldCondition, MatchValue

def retrieve_chunks(question: str, document_id: str, top_k: int = 4):
    query_vector = embeddings.embed_query(question)
    results = qdrant.search(
        collection_name=COLLECTION,
        query_vector=query_vector,
        query_filter=Filter(
            must=[FieldCondition(key="document_id", match=MatchValue(value=document_id))]
        ),
        limit=top_k,
    )
    return [
        {"chunk_text": r.payload["chunk_text"], "page_number": r.payload["page_number"]}
        for r in results
    ]
```

The metadata filter matters: without filtering by `document_id`, a question about Document A could retrieve chunks from Document B. This is a common bug in first-pass RAG implementations — mention it in interviews as something you specifically guarded against.

13. Answer Generation & Prompting

```
# app/core/prompts.py
SYSTEM_PROMPT = """You are a document assistant. Answer the question using
ONLY the context below. If the answer isn't in the context, say
"I couldn't find that in the document" — do not make anything up.

Context:
{context}
"""

# app/core/chain.py
from langchain_openai import ChatOpenAI
from app.core.retriever import retrieve_chunks
from app.core.prompts import SYSTEM_PROMPT

llm = ChatOpenAI(model=settings.chat_model, temperature=0)

def answer_question(question: str, document_id: str, chat_history: list[dict] = None):
    chunks = retrieve_chunks(question, document_id)
    context = "\n\n".join(f"[Page {c['page_number']}] {c['chunk_text']}" for c in chunks)

    messages = [{"role": "system", "content": SYSTEM_PROMPT.format(context=context)}]
    if chat_history:
        messages.extend(chat_history[-6:]) # last 3 turns for context
    messages.append({"role": "user", "content": question})

    response = llm.invoke(messages)
    return {"answer": response.content, "sources": chunks}
```

temperature=0 matters here — you want deterministic, grounded answers, not creative ones. Mention this as a deliberate choice, not a default you forgot to change.

14. FastAPI Routes

```
# app/routes/upload.py
from fastapi import APIRouter, UploadFile, BackgroundTasks
import uuid, shutil

router = APIRouter()

@router.post("/documents")
async def upload_document(file: UploadFile, background_tasks: BackgroundTasks):
    document_id = str(uuid.uuid4())
    path = f"/tmp/{document_id}.pdf"
    with open(path, "wb") as f:
        shutil.copyfileobj(file.file, f)

    background_tasks.add_task(process_document, document_id, path)
    return {"document_id": document_id, "status": "processing"}

def process_document(document_id: str, path: str):
    pages = extract_pages(path)
    chunks = chunk_pages(pages)
    store_chunks(document_id, chunks)
    # update Postgres status -> "ready"
```

```
# app/routes/ask.py
from fastapi import APIRouter
from app.schemas.schemas import AskRequest, AskResponse
from app.core.chain import answer_question

router = APIRouter()

@router.post("/ask", response_model=AskResponse)
async def ask(payload: AskRequest):
    result = answer_question(payload.question, payload.document_id, payload.chat_history)
    return AskResponse(**result, session_id=payload.session_id)
```

Why `BackgroundTasks` for upload? Embedding a large PDF can take seconds. Returning immediately with `status: processing` and letting the client poll `/documents/{id}` keeps the API responsive — same pattern real production upload pipelines use.

15. Chat History & Sessions

Store every message in Postgres so history survives restarts and can be replayed into the LLM context window.

```
# app/db/models.py
from sqlalchemy import Column, String, Text, TIMESTAMP, ForeignKey, JSON
from sqlalchemy.dialects.postgresql import UUID
from sqlalchemy.orm import declarative_base
import uuid, datetime

Base = declarative_base()

class Message(Base):
    __tablename__ = "messages"
    id = Column(UUID(as_uuid=True), primary_key=True, default=uuid.uuid4)
    session_id = Column(UUID(as_uuid=True), ForeignKey("sessions.id"))
    role = Column(String)          # 'user' | 'assistant'
    content = Column(Text)
    sources = Column(JSON, nullable=True)
    created_at = Column(TIMESTAMP, default=datetime.datetime.utcnow)
```

When answering, fetch the last few messages for that session and pass them into `answer_question()` as `chat_history` so follow-up questions like "what about clause 3?" resolve correctly.

16. Error Handling & Validation

Case	Handling
Non-PDF upload	Reject with 400 before saving to disk — check content-type and extension
Empty/scanned PDF (no extractable text)	Mark document status "failed" with a clear reason
Question asked before ingestion finishes	Return 409 Conflict — "document still processing"
OpenAI API failure/timeout	Catch, return 502 with a retry-safe error, log for debugging
Qdrant unreachable	Health check endpoint, fail fast with 503

```
from fastapi import HTTPException

if not file.filename.endswith(".pdf"):
    raise HTTPException(status_code=400, detail="Only PDF files are supported")
```

17. Wiring It Together (main.py)

```
# app/main.py
from fastapi import FastAPI
from app.routes import upload, ask, sessions

app = FastAPI(title="RAG Document Q&A System")

app.include_router(upload.router, tags=["documents"])
app.include_router(ask.router, tags=["qa"])
app.include_router(sessions.router, tags=["sessions"])

@app.get("/health")
def health():
    return {"status": "ok"}
```

```
uvicorn app.main:app --reload --port 8000
```

PART D

Shipping It

18. Docker & Docker Compose

```
# Dockerfile
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

```
# docker-compose.yml
services:
  api:
    build: .
    ports: ["8000:8000"]
    env_file: .env
    depends_on: [qdrant, postgres]

  qdrant:
    image: qdrant/qdrant
    ports: ["6333:6333"]
    volumes: ["qdrant_data:/qdrant/storage"]

  postgres:
    image: postgres:16
    environment:
      POSTGRES_DB: ragdb
      POSTGRES_PASSWORD: postgres
    ports: ["5432:5432"]
    volumes: ["pg_data:/var/lib/postgresql/data"]

volumes:
  qdrant_data:
  pg_data:
```

```
docker-compose up --build
```

19. Testing the API

```
# tests/test_ask.py
from fastapi.testclient import TestClient
from app.main import app

client = TestClient(app)

def test_ask_requires_document_id():
    response = client.post("/ask", json={"question": "test"})
    assert response.status_code == 422

def test_ask_returns_sources(mocked):
    mocked.patch("app.core.chain.answer_question", return_value={
        "answer": "test answer", "sources": [{"page_number": 1, "chunk_text": "..."}]
    })
    response = client.post("/ask", json={
        "document_id": "abc", "session_id": "xyz", "question": "test?"
    })
    assert response.status_code == 200
    assert "sources" in response.json()
```

Manual smoke test with curl:

```
curl -F "file=@sample.pdf" http://localhost:8000/documents
curl -X POST http://localhost:8000/ask \
-H "Content-Type: application/json" \
-d '{"document_id":"...", "session_id":"...", "question": "Summarize page 1"}'
```

20. Build Order Checklist

Follow top to bottom. Test after every milestone — don't write all the code first.

- ☐ Set up venv, install dependencies, create .env
- ☐ Spin up Qdrant + Postgres via docker-compose
- ☐ Write and test `extract_pages()` on one sample PDF
- ☐ Write and test `chunk_pages()` — print chunk count and sizes
- ☐ Write `store_chunks()`, verify points appear in Qdrant (dashboard at localhost:6333/dashboard)
- ☐ Write `retrieve_chunks()`, manually test with a known question
- ☐ Write the prompt template and `answer_question()`, test end-to-end from a script (no API yet)
- ☐ Wire up FastAPI upload + ask routes
- ☐ Add Postgres models for documents/sessions/messages
- ☐ Add chat history support, test a 2-turn follow-up conversation
- ☐ Add error handling for all cases in Chapter 16
- ☐ Write tests, get docker-compose fully working from a clean clone
- ☐ Write README with setup instructions and a demo GIF/screenshot
- ☐ Deploy (Render/Railway work well for this — Vercel doesn't run long-lived background workers well, keep that in mind for the ingestion background task)

21. Bugs You Will Hit

1. Embedding dimension mismatch. If you switch embedding models later, the Qdrant collection's vector size won't match. Fix: recreate the collection, or keep a version suffix on the collection name.

2. Empty answers from scanned PDFs. `extract_text()` returns "" silently. Add a check after extraction and fail the ingestion with a clear status message instead of silently indexing nothing.

3. Cross-document leakage. Forgetting the `document_id` filter in `retrieve_chunks()` means questions can pull chunks from unrelated documents. Always filter.

4. Context window overflow. Too many chunks or too much chat history sent to the LLM at once causes truncation or cost spikes. Cap `top_k` and history length explicitly (done above: `top_k=4`, last 6 messages).

5. Background task silently failing. FastAPI's `BackgroundTasks` swallow exceptions by default — wrap `process_document` in `try/except` and update status to "failed" so the client isn't left polling forever.

6. Duplicate uploads re-embedding the same content. Consider hashing the file content and checking Postgres before re-running the (paid) embedding step.

7. Qdrant collection not existing on first run. Race condition if two uploads hit `ensure_collection()` at once. Create the collection once at app startup instead of per-request.

22. Interview Q&A

Q: Why Qdrant over Pinecone or a plain FAISS index?

A: Open source and self-hostable (no vendor lock-in, no per-query cost), supports metadata filtering natively which FAISS doesn't out of the box, and has a simple Docker setup suitable for a self-contained demo.

Q: How do you prevent hallucination?

A: The system prompt explicitly restricts the model to the provided context and instructs it to say when the answer isn't present. Temperature is set to 0 for determinism. This isn't a hard guarantee, but it meaningfully reduces fabrication compared to an unconstrained prompt.

Q: How would you scale this to thousands of documents?

A: Qdrant scales horizontally via sharding; the main cost driver becomes embedding calls, which could be batched and cached. I'd also move ingestion to a proper task queue (Celery/RQ) instead of FastAPI's in-process BackgroundTasks, which don't survive a server restart.

Q: What's your chunk size and why?

A: 800 characters with 150 overlap — small enough to keep each chunk semantically focused for accurate retrieval, with overlap to avoid losing context at boundaries. This is a tunable parameter I'd adjust based on document type (legal text vs. narrative prose behave differently).

Q: How do you evaluate RAG quality?

A: Manually curated question/expected-answer pairs against known documents, checking both retrieval precision (are the right chunks coming back) and answer faithfulness (is the answer actually supported by the retrieved context). For production I'd mention RAGAS or similar automated eval frameworks as the next step.

Q: What's the difference between this and just increasing the LLM's context window?

A: Even with huge context windows, stuffing entire documents in is expensive per-call and the model's attention degrades over very long contexts ("lost in the middle"). Retrieval keeps only the relevant slice, which is cheaper and more accurate.

23. Cheatsheet

Task	Command / Snippet
Start everything	<code>docker-compose up --build</code>
Run API only (local dev)	<code>uvicorn app.main:app --reload</code>
Qdrant dashboard	<code>http://localhost:6333/dashboard</code>
API docs	<code>http://localhost:8000/docs</code>
Run tests	<code>pytest tests/ -v</code>
Embed a query	<code>embeddings.embed_query(text)</code>
Search Qdrant	<code>qdrant.search(collection_name, query_vector, limit)</code>

Environment variables recap

```
OPENAI_API_KEY
QDRANT_URL
DATABASE_URL
EMBEDDING_MODEL
CHAT_MODEL
```

Next: once this is deployed with a live URL and pushed to GitHub with a clean README, move on to Project 2 (NL-to-SQL Generator) — it reuses this exact LangChain + OpenAI setup, swapping the retrieval target from document chunks to a database schema.